```
$newCars = array_map(function (array $car, int $value): array {
    return [
        $car[0][0] => $car[0][1],
        $car[1][0] => $car[1][1],
        'value' => $value,
    ];
}, $cars, $values);

print_r($newCars);

Output:
Array
(
    [0] => Array
        (
            [make] => VW
            [model] => Golf GTI
            [value] => 20000
        )

    [1] => Array
        (
            [make] => VW
            [model] => Corrado VR6
            [value] => 12000
        )

)
```

`array_map()` takes a minimum of two arguments: the callback to be executed for each element and an array. It can optionally take further arrays: each iteration of the callback will be given the n-th value of each array, meaning you can work on multiple arrays in parallel – just make sure the number of elements in each is the same! Here we are combining the data from two arrays and condensing them into a single new array, and this time we're using an anonymous function rather than a named one. Although it is not shown here, examining the contents of the original arrays would show that they retain their original values.